

Витрина комиссий NEW

Платформа налоговых данных

Exported on 01/13/2026

Table of Contents

1	Описание	6
2	Режимы запуска.....	7
3	Входные и выходные данные.....	8
4	Используемые таблицы	9
4.1	Источники	9
4.2	Справочники	10
4.3	Таблицы подписки	10
5	Шаги процесса	14
6	Подготовительные шаги	15
6.1	Получить входные параметры	15
6.2	Определить константы и переменные для использования в алгоритме	15
6.3	Определить период загрузки.....	15
6.4	Получить значения настроек из настроечных таблиц	15
6.5	Определить пути к таблицам.....	15
6.6	Заполнить датасет Подразделение-Филиал (vspc)	15
6.7	Получить список ВЗЛ и список ОЗ за период загрузки из tax_dictionary	16
6.7.1	1. Получение списка ВЗЛ за период из сервиса tax-dictionary.....	16
6.7.2	2. Получение списка ОЗ (оффшорных зон) за период из сервиса tax-dictionary.....	20
6.7.2.1	2.1. Получение основных оффшорных территорий:	20
6.7.2.2	2.2. Получение частей оффшорных территорий:	22
6.7.2.3	2.3. Алгоритм получения ОЗ действующих за требуемый период.....	26
7	Отбор данных.....	28
7.1	Выбираем проводки из реплики ГБК за период загрузки	28
7.2	Добавим ID проводки в ЕКС/gl_gl для сверки с проводками для РНУ-4,5.....	31
7.3	Добавим сумму исправительных проводок	32
7.4	Ищем операции в реплике УДКЗ.....	34
7.5	Получаем данные договора из УДКЗ	36
7.6	Ищем операции в реплике Биллинга	36
7.7	Получаем данные договора из реплики ППРБ-источника или из ЕСС	37

7.8	Добавляем ИНН, наименование и прочие данные клиента.....	40
7.9	Добавляем данные о ВЗЛ и РОЗ.....	41
7.10	Добавляем код филиала.....	42
7.11	Исключаем ИП и НКЦ.....	43
7.12	Заносим в таблицу витрины.....	43
8	Заполнение данных в таблице витрины.....	44
8.1	Заполнение полей таблицы.....	44
8.2	Запись данных в таблицу отчета.....	51
9	Требования к нефункциональным характеристикам	53
9.1	Требования к журналированию	53
9.2	Требования к частоте и периодичности загрузки.....	53
9.3	Требование к объему и времени загрузки	53
9.4	Требование к первой загрузке данных	53

История изменений

Дата	Автор	Описание	Задача Jira на аналитику	Задача Jira на разработку
02.04.2025	Морозова Е.	Документ создан	✓ PND-192 ¹ - А_ВД_Аналитика для Витрины комиссий (ЕКС) CLOSED	
17.06.2025	Морозова Е.	Добавлен новый способ связи проводки с регистром в УДКЗ	✓ PND-814 ² - А_ВД_Доработка витрины комиссий по результатам parallel run CLOSED	
23.07.2025	Бычков Д.	Добавлен субтип контрагента (исключены ИП) и связь с договором РКО и контрагентом через номер счета для случаев отсутствия связи с УДКЗ и билингом	✓ PND-2545 ³ - А_ВД_Развитие витрины комиссий CLOSED	
30.07.2025	Бычков Д.	Добавлена связь с договорами и контрагентами в УДКЗ для счетов 47422, связь с договорами и контрагентами в КЮЛ для счетов 47441, исключены счета НКО НКЦ	✓ PND-2599 ⁴ - А_ВД_Развитие витрины комиссий (Бизнес-карта) CLOSED	

¹ <https://jira.sberbank.ru/browse/PND-192?src=confmacro>

² <https://jira.sberbank.ru/browse/PND-814?src=confmacro>

³ <https://jira.sberbank.ru/browse/PND-2545?src=confmacro>

⁴ <https://jira.sberbank.ru/browse/PND-2599?src=confmacro>

Дата	Автор	Описание	Задача Jira на аналитику	Задача Jira на разработку
01.09.2025	Бычков Д.	Добавлены поля serviceid , productRegister , register_status , скорректированы фильтры при связи с УДКЗ	✓ PND-2626 ⁵ - А_ВД_Развитие витрины комиссий (УДКЗ+новые поля) CLOSED	
31.10.2025	Бычков Д.	Добавлена связь асст с subj для счетов напротив 706%	✓ PND-3121 ⁶ - А_ВД_Добавление связи счет-контрагент в витрину комиссий для Приложений 6.4, 6.23, 6.24 CLOSED	📌 PND-3141 ⁷ - Т_ВД_Добавление связи счет-контрагент в витрину комиссий для Приложений 6.4, 6.23, 6.24 TO DO
01.11.2025	Бычков Д.	Добавлена связь с целевым БУ	✓ PND-3120 ⁸ - А_ВД_Добавление целевого БУ в витрину комиссий для Приложений 6.4, 6.23, 6.24 CLOSED	📌 PND-3142 ⁹ - Т_ВД_ИС Добавление новых целей оптимизации DONE

⁵ <https://jira.sberbank.ru/browse/PND-2626?src=confmacro>

⁶ <https://jira.sberbank.ru/browse/PND-3121?src=confmacro>

⁷ <https://jira.sberbank.ru/browse/PND-3141?src=confmacro>

⁸ <https://jira.sberbank.ru/browse/PND-3120?src=confmacro>

⁹ <https://jira.sberbank.ru/browse/PND-3142?src=confmacro>

1 Описание

Страница содержит описание процедуры заполнения витрины комиссий на основе данных бух. учета.

- Сначала отбираются проводки на счет доходов по списку символов ОФР (комиссии),
- потом в УДКЗ и в биллинге (позже, возможно, еще где-то) по проводкам ищем продуктовую операцию,
- потом из фабрики, породившей комиссию, получаем номер и дату договора.

2 Режимы запуска

Предполагается запуск по расписанию: еженедельно или ежедневно.

- запуск по расписанию делается без указания периода загрузки: заполнение происходит за последние N дней
 - N = 14 пока. (⚠ уточнить)

Возможен запуск вручную (по требованию), с указанием дат начала и окончания периода загрузки.

Партиции в таблице отчета - по дата-продукту и дате проводки в бухучете.

3 Входные и выходные данные

Входные параметры

Параметр	Описание	Обязательность	Ограничения	Комментарий
\$Дата-продукт	Код дата-продукта - источника данных	Н	Допустимые значения: repl_gbk_stream	
\$start_dt	Дата начала периода загрузки	Н	- Поля одновременно заданы или не заданы. \$start_dt не больше \$end_dt \$end_dt не больше текущей системной даты	Если период не задан, заполняем за последние N дней N задано в таблице DATA_PRODUCT ¹⁰ в поле transaction_depth_update по умолчанию 14 дней
\$end_dt	Дата окончания периода загрузки	Н		

Входные данные

- Таблицы подписки: реплика ГБК, реплика УДКЗ-4, ППРБ Биллинг, ЕСС. РКО ЮЛ...
 - см. [Используемые таблицы](#)(see page 9)
- Константы (системные настройки)
 - см. [Константы](#)¹¹
- Таблицы настроек:
 - см. [Используемые таблицы. Справочники](#)(see page 10)
- Справочники tax-dictionary (вызов сервисов)

Выходные данные

- Заполненная таблица отчета: [TAXDM_STG_COMMISSION](#)¹².

¹⁰ https://confluence.sberbank.ru/display/PND/DATA_PRODUCT

¹¹ <https://confluence.sberbank.ru/pages/viewpage.action?pageId=6043895168>

¹² https://confluence.sberbank.ru/display/PND/TAXDM_STG_COMMISSION

4 Используемые таблицы

4.1 Источники

- Реплика ГБК новая (**platform_gbk_stream**) - далее **gbk**
 - Структура таблиц - [platform_gbk_stream](#). Структура таблиц подписки¹³
- Реплика ГБК mi (**platform_gbkmi_stream**) - далее **gbkmi**
 - Структура таблиц - [platform_gbkmi_stream](#). Структура таблиц подписки¹⁴
- Реплика УДКЗ 4gen - далее **udkz**
 - Структура таблиц - [УДКЗ 4gen](#). Подписка на реплику. Структура таблиц¹⁵
 - Описание таблиц БД: [Описание БД УДКЗ-4 \(Cloud\)](#)¹⁶
- Реплика ППРБ Зарплатные договоры (**platform_pprbzdfd**) - далее **zdfd**
 - Структура таблиц - [platform_pprbzdfd](#). ППРБ ЮЛ Зарплатные договоры. Структура таблиц подписки¹⁷
 - Страница с информацией о реплике: Зарплатные договоры¹⁸
 - Жизненный цикл зарплатного/социального договора: <https://confluence.delta.sbrf.ru/pages/viewpage.action?pagelId=12321692867>
- Реплика ППРБ Биллинг - далее **billing**
 - Структура таблиц - [platform_billing_calc](#). ППРБ Биллинг. Структура таблиц подписки¹⁹
- Реплика ППРБ АЕ
 - [Структура таблиц](#) - Реплика ППРБ АЕ. INTERNAL_AEACC_AEACC. Структура таблиц подписки²⁰
- **settl_serv_org_idl_debug** (дата-продукт ndl_p_rko_c_rko) - далее **serv_org**
 - [settl_serv_org_idl_debug](#). Структура таблиц подписки²¹
- **corp_cgc_ndl_p_kul**
 - [corp_cgc_ndl_pkul](#). Структура таблиц подписки²²
- **subj_org_idl** - далее **subj_org**
 - [subj_org_idl](#). Структура таблиц подписки²³
- **gl_idl**
 - [gl_idl](#). Структура таблиц подписки²⁴
- **gl_idl1**
 - [gl_idl1](#) (БУ FESS4). Структура таблиц подписки²⁵
- **vspc_idl**
 - [vspc_idl](#). Структура таблиц подписки²⁶
- **rdm_idl_spark**

¹³ <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=16277773361>

¹⁴ <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=19063412608>

¹⁵ <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=19063407835>

¹⁶ <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=17355200968>

¹⁷ <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=19063408832>

¹⁸ <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=14808842526>

¹⁹ <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=19063410578>

²⁰ <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=20430621377>

²¹ <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=17698627809>

²² <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=19721691385>

²³ <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=10025109036>

²⁴ <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=12764489926>

²⁵ <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=21269975595>

²⁶ <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=15807981221>

- [rdm_idl_spark. Структура таблиц подписки](#)²⁷

4.2 Справочники

Список справочников

Справочник	Описание	Комментарий
Дата-продукты - источники данных data_product	DATA_PRODUCT ²⁸	добавить: 1. repl_udkz4g 2. repl_zdfd 3. repl_gbkmi_stream
Маппинг филиалов витрин	TECHDM_FILIAL_MAP ²⁹	
Настройки отбора для витрины проводок СФ	TSO_APP_SETTING_TRANSACTION_FILTER ³⁰	ранее были добавлены записи для 6.4 (см. Структура БД. Приложение 6.4 ³¹)

4.3 Таблицы подписки

Используемые таблицы

Источник	Таблица	Описание	Комментарий
repl_gbk_stream	com_sbt_gbk_standin_transport_dto_ Transaction DTO далее Transaction	Полупроводки ГБК	

²⁷ <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=10063645226>

²⁸ https://confluence.sberbank.ru/display/PND/DATA_PRODUCT

²⁹ https://confluence.sberbank.ru/display/PND/TECHDM_FILIAL_MAP

³⁰ https://confluence.sberbank.ru/display/PND/TSO_APP_SETTING_TRANSACTION_FILTER

³¹ <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=11161963855#id->

СтруктураБД.Приложение6.4-TSO_APP_SETTING_TRANSACTION_FILTER

Источник	Таблица	Описание	Комментарий
repl_gbk_stream	com_sbt_gbk_standin_transport_dto_ AccountingDocument DTO далее AccountingDocument	Учетные документы (проводки) ГБК	
repl_gbk_stream	com_sbt_gbk_standin_transport_dto_ Account DTO далее Account	Аналитический счет ГБК	
repl_gbkmi_stream	com_sbt_gbk_integration_standin_dto_ EksGbkLink DTO далее EksGbkLink	Связь проводок ЕКС и ГБК	
repl_udkz4g	register	Регистры	
repl_udkz4g	product	Продукт	в УДКЗ продукт - это не договор. Это, похоже, одно событие начисления комиссии.
repl_udkz4g	filter_element_attribute	Доп. инфо по продукту	
repl_udkz4g	product_service_ids	Связь продукт - ID сервиса	
repl_udkz4g	document	Документы источника	
repl_zdfd	com_sbt_salary_agreement_model_oracle_ salaryagreementcard далее salaryAgreementCard	Версии договора	
repl_billing	ru_sber_billing_calregistry_model_ operation (далее wo_operation)	Биллинг. Списание комиссии	

Источник	Таблица	Описание	Комментарий
repl_billing	ru_sber_billing_calregistry_model_ pprboperation (далее calc_operation)	Биллинг. Расчет комиссии	
INTERNAL_AEACC_AEACC	account	Реплика AE	
subj_org_idl	subj	Субъекты ЮЛ	
subj_org_idl	subj_h	Субъекты ЮЛ - сателлит	
subj_org_idl	subj_orgidfn	Субъекты ЮЛ - сателлит	
rdm_idl_spark	cntry	Страны	
gl_idl	acct	Аналитические счета	
gl_idl	acct_h	Аналитические счета - сателлит	
gl_idl	trn_trn	Связи "проводка - корректирующая проводка"	Временное решение: берем связи из дата-продукта gl_ndl_eks, поскольку для ускорения работы корректирующие проводки выбираем из tax_transaction_rnu_4_5 ³² (Т1 для РНУ-4,5, сейчас заполняется по gl_ndl_eks). После перевода РНУ-4,5 на источник gl_gl здесь тоже надо будет перейти.
gl_idl1	acct	Аналитические счета	

32 https://confluence.sberbank.ru/display/PND/tax_transaction_rnu_4_5

Источник	Таблица	Описание	Комментарий
gl_idl1	acct_h	Аналитические счета - сателлит	
vspc_idl	div_	Подразделения территориальной структуры	
settl_serv_org_idl_debug	agr	Договоры (РКО ЮЛ)	
	registry	Регистры (РКО ЮЛ)	
corp_cgc_ndl_p_kul	agr	Договоры (КЮЛ)	
	registry	Регистры (КЮЛ)	

5 Шаги процесса

№	Шаг	Описание	Входные данные	Результат
1	Подготовительные шаги	1. Получить входные параметры 2. Определить даты периода загрузки 3. Получить и проверить входные данные 4. Подготовить вспомогательные датасеты	Константы Справочники витрин Справочники tax-dictionary (вызов сервисов)	Получены входные параметры Определен период загрузки Определены пути к таблицам Подготовлены датасеты: 1. Подразделение - тербанк 2. ВЗЛ и ОЗ.
2	Отбор данных	Чтение данных из таблиц подписки	Таблицы подписки(see page 10)	Датасет
3	Запись в таблицу витрины	Замена данных в таблице : 1. Удалить партиции по \$Дата-продукту за период загрузки. 2. Записать строки датасета в таблицу витрины.	Датасет Таблица TAXDM_STG_COMMISSION ³³ (возможно, со старыми данными)	Таблица TAXDM_STG_COMMISSION ³⁴ с актуальными данными за период загрузки

³³ https://confluence.sberbank.ru/display/PND/TAXDM_STG_COMMISSION

³⁴ https://confluence.sberbank.ru/display/PND/TAXDM_STG_COMMISSION

6 Подготовительные шаги

6.1 Получить входные параметры

Получить и проверить входные параметры (см. п. 3)

6.2 Определить константы и переменные для использования в алгоритме

При описании алгоритма используются следующие значения ( здесь и далее описана логика, конкретная реализация - на усмотрение разработчика):

- `fd.taxdm_stg_commisson.product_fabric_code_exclude` - Коды продуктовых фабрик, исключаемые из отбора в витрину комиссий

6.3 Определить период загрузки

Если параметры `$start_dt` (дата начала периода загрузки), `$end_dt` (дата окончания периода загрузки) не переданы, вычисляем их:

- `$start_dt = T - transaction_depth_update` из таблицы `DATA_PRODUCT`³⁵ для \$Дата-продукта
- `$end_dt = T - 1`

где T - текущая дата.

6.4 Получить значения настроек из настроечных таблиц

Получаем **фильтры по маскам счетов** из таблицы `TSO_APP_SETTING_TRANSACTION_FILTER`³⁶

- `tax_doc_type = 'tso_64'`
- `start_date <= $end_dt <= end_date`

6.5 Определить пути к таблицам

Определить пути ко всем используемым таблицам (см. п. [Используемые таблицы](#) (see page 9)).

6.6 Заполнить датасет Подразделение-Филиал (vsprc)

 Здесь используем алгоритм, который уже был реализован для `TECHDM_TRANSACTION`. Загрузка проводок (1 Этаж)³⁷ и других наших ETL-алгоритмов. Описание привожу для полноты картины, необходимо переиспользовать готовый код.

³⁵ https://confluence.sberbank.ru/display/PND/DATA_PRODUCT

³⁶ https://confluence.sberbank.ru/display/PND/TSO_APP_SETTING_TRANSACTION_FILTER

³⁷ <https://confluence.sberbank.ru/pages/viewpage.action?pageId=9409962354>

Делаем датасет "Подразделение-Филиал" (далее **vspc**) по таблицам дата-продукта `vspc_idl`:

1. Джойним `div_` саму с собой (далее `div_`, `div_parent`):
 - a. По условию `div_.parent_div_sk = div_parent.div_sk`
 - b. рекурсивно, пока не встретим запись, у которой `div_parent.parent_div_sk = NULL` - это будет ПАО Сбербанк, уровень 0.
 - c. нам нужны данные подразделения уровня 1 (тер. банка):
 - i. `div_.div_sk`
 - ii. `div_.subbranch_code`
2. Заносим в датасет для каждой записи исходной `div_` :
 - a. `div_sk` исходной записи
 - b. `terr_bank_div_sk := div_.div_sk` найденного тер.банка
 - c. `terr_bank_subbranch_code := div_.subbranch_code` найденного тер.банка
3. Если у исходной записи `parent_div_sk = NULL` (ПАО Сбербанк), запишем:
 - a. `terr_bank_div_sk := div_sk`
 - b. `terr_bank_subbranch_code := subbranch_code`

6.7 Получить список ВЗЛ и список ОЗ за период загрузки из `tax_dictionary`

Получение ВЗЛ

6.7.1 1. Получение списка ВЗЛ за период из сервиса `tax-dictionary`

- Метод **GET /v1/interdependent_entity/interdependent_entity**
- Параметры запроса:
 - **page_num** - число, номер страницы, нумерация с 0
 - **page_size** - число, количество строк для отображения на странице
- Параметры ответа:
 - **Ответ JSON**

```
{
  "page_info": {
    "page_token": "",
    "total_size": 100,
    "total_page": 0
  },
  "interdependent_entity": [
    {
      "code": "string",
      "name": "string",
      "full_russian_name": "string",
      "full_latin_name": "string",
      "address": "string",
      "rus_fed_resident_sign": true,
      "country_code": "string",
      "inn": "string",
      "kpp": "string",
      "swift_code": "string",
      "inc_country_reg_number": "string",

```

```

    "list_inclus_reason_date": "2020-01-01",
    "list_exclus_reason_date": "2020-01-01",
    "traf_light_categ_code": {
      "code": "string",
      "description": "string"
    },
    "vat_taxpayer_sign": false,
    "special_tax_regime_code": {
      "code": "string",
      "description": "string"
    },
    "interdepend_criterion": {
      "code": "string",
      "name": "string",
      "description": "string"
    }
  }
]
}

```

- **Описание полей ответа**

PageInfo блок опустим.

Уровень 1	Уровень 2	Уровень 3	Обязательность	Описание
interdependent_entity				Массив Взаимозависимых лиц Сбербанка из таблицы interdependent_entity
	code		+	Идентификационный код
	name		+	Наименование ЮЛ
	full_russian_name		+	Полное наименование ЮЛ на русском языке
	full_latin_name		-	Полное наименование ЮЛ латиницей
	address		+	Местонахождение юридического лица

Уровень 1	Уровень 2	Уровень 3	Обязательность	Описание
	rus_fed_resident_sign		+	Признак отнесения организации к резидентам РФ
	country_code		-	Код страны по ОКМ
	inn		-	ИНН
	kpp		-	КПП
	swift_code		-	SWIFT
	inc_country_registration_number		-	Регистрационный номер в стране инкорпорации
	list_inclusion_reason_date		+	Дата наступления основания для включения в список
	list_exclusion_reason_date		-	Дата наступления основания для исключения из списка
	traff_light_category_code		+	Объект "Категория взаимозависимости по системе Светофор"
		code	+	Код категории
		description	+	Описание категории
	vat_taxpayer_sign		+	Признак налогоплательщика по НДС

Уровень 1	Уровень 2	Уровень 3	Обязательность	Описание
	special_tax_regime_code		+	Объект "Применимость особого режима налогообложения"
		code	+	Код применимости особого режима налогообложения
		description	+	Описание применимости особого режима налогообложения
	interdependent_criterion		+	Объект "Критерий взаимозависимости с ПАО Сбербанк"
		code	+	Код критерия
		name	+	Наименование критерия
		description	+	Описание критерия

- Алгоритм получения ВЗЛ действующих за требуемый период. Так как метод возвращает весь справочник без возможности отбора ВЗЛ действующих в требуемом периоде, то отбор лиц, действующих за требуемый период необходимо реализовать на нашей стороне. Порядок действий следующий:
 - Постранично прочитать весь справочник.
 - Отобрать элементы массива **interdependent_entity** где, **list_inclus_reason_date** и **list_exclus_reason_date** входят в интервал требуемого периода. см. ниже примеры отбора,
 - Далее датасет будет называться как - **interdependent_entity_data**

Примеры отбора

Требуемый период 9 мес. 2022г. т.е. 01.01.2022 по 30.09.2022

list_inclus_reason_date	list_exclus_reason_date	Результат
01.01.2021	31.12.2021	Отбрасываем
01.01.2021	null	Включаем
01.01.2021	30.06.2022	Включаем
01.05.2022	null	Включаем
01.05.2022	31.12.2022	Включаем
30.11.2022	null	Отбрасываем
01.01.2022	30.09.2022	Включаем

Получение списка ОЗ

6.7.2 2. Получение списка ОЗ (оффшорных зон) за период из сервиса tax-dictionary

6.7.2.1 2.1. Получение основных оффшорных территорий:

- Метод **GET /v1/offshore_state/offshore_state_ter**
- Параметры запроса:
 - **page_num** - число, номер страницы, нумерация с 0
 - **page_size** - число, количество строк для отображения на странице
- Параметры ответа:
 - **Ответ Json**

```
{
  "page_info": {
    "page_token": "",
    "total_size": 100,
    "total_page": 0
  },
  "offshore_state_ter": [
    {
      "code": 0,
      "position_index": 0,
      "name": "string",
      "country_code": "string",
      "rel_point_entirely_sign": true,

```

```

    "effective_order_code": {
      "code": 0,
      "order_number": "string",
      "order_signing_date": "2020-01-01",
      "order_effective_date": "2020-01-01",
      "description": "string"
    },
    "termination_order_code": {
      "code": 0,
      "order_number": "string",
      "order_signing_date": "2020-01-01",
      "order_effective_date": "2020-01-01",
      "description": "string"
    },
    "description": "string"
  },
]
}

```

- **Описание полей ответа**

PageInfo блок опустим.

Уровень 1	Уровень 2	Уровень 3	Обязательность	Описание
values				Массив частей оффшорных зон Сбербанка из таблицы offshore_state_ter
	code		+	Код оффшорной зоны
	position_index		+	Номер пункта по порядку
	name		+	Наименование оффшорной зоны
	country_code		+	Код страны по ОКСТ
	rel_point_entirely_sign		+	Признак отнесения пункта Приказа целиком к оффшорной зоне

Уровень 1	Уровень 2	Уровень 3	Обязательность	Описание
	effective_order_code		+	Объект "Приказ Минфина"
		code	+	Код приказа
		order_number	+	Номер приказа
		order_signing_date	+	Дата подписания приказа
		order_effective_date	+	Дата вступления в силу приказа
		description	+	Описание
	termination_order_code		-	Объект "Приказ Минфина"
		code	+	Код приказа
		order_number	+	Номер приказа
		order_signing_date	+	Дата подписания приказа
		order_effective_date	+	Дата вступления в силу приказа
		description	+	Описание
	description		-	

6.7.2.2 2.2. Получение частей оффшорных территорий:

- Метод GET /v1/offshore_state/offshore_state_ter_part
- Параметры запроса:
 - **page_num** - число, номер страницы, нумерация с 0
 - **page_size** - число, количество строк для отображения на странице

- Параметры ответа:

- Ответ Json

```
{
  "values": [
    {
      "offshore_code": {
        "code": 0,
        "position_index": 0,
        "name": "string",
        "country_code": "string",
        "rel_point_entirely_sign": true,
        "effective_order_code": {
          "code": 0,
          "order_number": "string",
          "order_signing_date": "2023-06-23",
          "order_effective_date": "2023-06-23",
          "description": "string"
        },
        "termination_order_code": {
          "code": 0,
          "order_number": "string",
          "order_signing_date": "2023-06-23",
          "order_effective_date": "2023-06-23",
          "description": "string"
        },
        "description": "string"
      },
      "code": 0,
      "name": "string",
      "country_code": "string",
      "effective_order_code": {
        "code": 0,
        "order_number": "string",
        "order_signing_date": "2023-06-23",
        "order_effective_date": "2023-06-23",
        "description": "string"
      },
      "termination_order_code": {
        "code": 0,
        "order_number": "string",
        "order_signing_date": "2023-06-23",
        "order_effective_date": "2023-06-23",
        "description": "string"
      },
      "description": "string"
    }
  ],
  "page_info": {
    "page_token": "string",
    "total_size": 0
  }
}
```

```

    }
  }

```

- **Описание полей ответа**

PageInfo блок опустим.

Уровень 1	Уровень 2	Уровень 3	Обязательность	Описание
offshore_code				 Объект родительской территории. Требуется использовать только поле offshore_code → code для исключения родительских зон.
code		+	Код части офшорной зоны	
name		+	Наименование части офшорной зоны	
country_code		+	Код страны по ОКСМ	
effective_order_code		+	Объект "Приказ Минфина"	
	code	+	Код приказа	
	order_number	+	Номер приказа	

Уровень 1	Уровень 2	Уровень 3	Обязательность	Описание
	order_signing_date	+	Дата подписания приказа	
	order_effective_date	+	Дата вступления в силу приказа	
	description	+	Описание	
termination_order_code		-	Объект "Приказ Минфина"	
	code	+	Код приказа	
	order_number	+	Номер приказа	
	order_signing_date	+	Дата подписания приказа	
	order_effective_date	+	Дата вступления в силу приказа	
	description	+	Описание	
description		-	Описание	

6.7.2.3 2.3. Алгоритм получения ОЗ действующих за требуемый период.

- Так как методы возвращают весь справочник без возможности отбора ОЗ действующих в требуемом периоде, то отбор зон, действующих за требуемый период необходимо реализовать на нашей стороне.
- Порядок действий следующий:
 - Постранично прочитать справочник. `/v1/offshore_state/offshore_state_ter`
 - Отобразить элементы массива `values` где, `effective_order_code` → `order_effective_date` и `termination_order_code` → `order_effective_date` входят, не включая правую границу (`termination_order_code` → `order_effective_date`) в интервал требуемого периода см. ниже примеры отбора.
 - Добавить в датасет (далее будет называться как `offshore_state_ter_data`)
 - Постранично прочитать справочник. `/v1/offshore_state/offshore_state_ter_part`
 - Отобразить элементы массива `values` где, `effective_order_code` → `order_effective_date` и `termination_order_code` → `order_effective_date` входят, не включая правую границу (`termination_order_code` → `order_effective_date`) в интервал требуемого периода см. ниже примеры отбора.
 - Добавить в датасет (далее будет называться как `offshore_state_ter_data`)
 - Преобразования датасета:
 - Исключить территории из массива `/v1/offshore_state/offshore_state_ter`, которые имеют дочерние территории в `/v1/offshore_state/offshore_state_ter_part` (связь родительской и дочерней территории осуществляется через поля `offshore_state_ter` → `values` → `code` И `offshore_state_ter_part` → `values` → `offshore_code` → `code`).
 - По записям, где `offshore_state_ter_data.country_code = 826` и `rel_point_entirely_sign = true` оставить одну любую запись. (На момент постановки их 3 "Соединенные королевства великобритании и северной ирландии", "Суверенные базовые районы акротир и декелия" (входит в состав, но не часть), "британская антарктическая территория" (входит в состав, но не часть)).

Примеры отбора

Требуемый период 9 мес. 2022г. т.е. 01.01.2022 по 30.09.2022

<code>effective_order_code</code> → <code>order_effective_date</code>	<code>termination_order_code</code> → <code>order_effective_date</code>	Результат
01.01.2021	31.12.2021	Отбрасываем
01.01.2021	null	Включаем
01.01.2021	30.06.2022	Включаем

effective_order_code → order_effective_date	termination_order_code → order_effective_date	Результат
01.05.2022	null	Включаем
01.05.2022	31.12.2022	Включаем
30.11.2022	null	Отбрасываем
30.11.2021	01.01.2022	Отбрасываем
01.01.2022	30.09.2022	Включаем

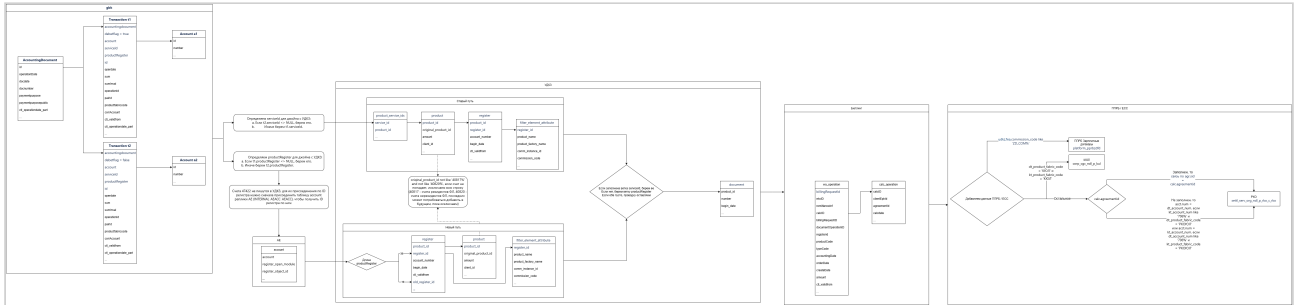
⚠ Сейчас чтение из двух источников не оптимально, задан вопрос в tax-dictionary на вывод территорий и её частей в одном ответе.

7 Отбор данных

Раздел описывает шаг 2 процесса загрузки.

Описание таблиц и их используемых полей дано в п.4.

Схема алгоритма



7.1 Выбираем проводки из реплики ГБК за период загрузки

1. Выбираем проводки из таблиц реплики **gbk** за период загрузки, удовлетворяющие фильтру по маскам счетов Дт и Кт:
 - a. Джойн таблиц:
 - i. **AccountingDocument** (далее **ad**)
 - ii. **Transaction** - 2 раза (**t1** - дебет, **t2** - кредит)
 - iii. **Account** - 2 раза (**a1** - дебет, **a2** - кредит)
 - b. Условие джойна:
 - i. accountingdocument **ad**
 - ii. join transaction **t1** on t1.accountingdocument = ad.id and t1.debetflag = true -- полупроводка дебет
 - iii. join account **a1** on a1.id = t1.account
 - iv. join transaction **t2** on t2.accountingdocument = ad.id and t2.debetflag = false -- полупроводка кредит
 - v. join account **a2** on a2.id = t2.account
 - c. Фильтр:
 - i. \$start_dt <= t1.ctl_operdate_part <= \$end_dt -- это целевой отбор по дате, остальные фильтры по датам партий - для оптимизации
 - ii. \$start_dt <= t2.ctl_operdate_part <= \$end_dt
 - iii. \$start_dt <= ad.ctl_operationdate_part <= \$end_dt
 - iv. t1.pairid = t2.pairid -- можно в условие джойна
 - v. Фильтр по продуктовой фабрике - источнику:
 1. t1.productfabriccode не в списке ('ТКП', 'ЦОД', 'ЕКП', 'УВХД', 'ЗР', NULL) **или** -- константа fd.taxdm_stg_commisson.product_fabric_code_exclude
 2. t2.productfabriccode не в списке ('ТКП', 'ЦОД', 'ЕКП', 'УВХД', 'ЗР', NULL). -- та же константа
 3.  NULL в списке означает, что в условие фильтра надо добавить сравнение "is not null"

4.  Аналогичный парсинг списка и отбор по коду источника уже реализован в тех. витрине проводок в рамках задачи PND-1725 - см. [TECHDM_TRANSACTION. Загрузка проводок \(1 Этаж\)](#)³⁸
5.  С указанным выше списком фильтр может выглядеть так:
 - a. `t1.productfabriccode not in ('ТКП', 'ЦОД', 'ЕКП', 'УВХД') and t1.productfabriccode is not NULL` **or**
 - b. `t2.productfabriccode not in ('ТКП', 'ЦОД', 'ЕКП', 'УВХД') and t2.productfabriccode is not NULL.`
- vi. Фильтр по фабрике - владельцу счета:
 1. `a1.openingModule` не в списке ('ТКП', 'ЦОД', 'ЕКП') **или** -- константа `fd.taxdm_stg_commisson.account_opening_module_exclude`
 2. `a2.openingModule` не в списке ('ТКП', 'ЦОД', 'ЕКП'). -- та же константа
- vii. **Фильтр по маскам счета** из настроек:
 1. `a1.number` - номер счета Дт
 2. `a2.number` - номер счета Кт

3. Алгоритм отбора проводок - подробнее

Читаем все строки настроек из таблицы "[Настройки отбора операций для приложений ТЦО](#)"³⁹⁴⁰ для \$Отчетная форма, актуальные на дату окончания периода загрузки \$end_dt.

Строка настроек включает следующие поля:

- Отчетная форма (\$Отчетная форма)
- Маски счетов Дт и Кт - могут иметь вид:
 - Одна маска (строка, включающая % и/или "?", подробнее см. [Правила ввода масок в настроечных таблицах](#)⁴¹⁴²);
 - 2 или более масок через запятую;
 - Пустая строка (NULL, пробел, отсутствие заполнения трактуем как пустую строку)
- Период актуальности (выбираем строки, где дата окончания периода загрузки попадает в период актуальности).

Заполняем "*условие фильтра*" (логическая сущность, используется исключительно для упрощения формулирования требований. Реализация на усмотрение разработчика):

- a. Для каждой отобранной строки таблицы настроек составить Условие:
 - i. Перекодировать Маски Дт и Кт из пользовательского формата, как указано в [Правила ввода масок в настроечных таблицах](#)⁴³⁴⁴.
 - ii. Разобрать Маски Дт и Кт - выделить части, разделенные запятыми:
 - i. МаскаДт1, МаскаДт2, ...
 - ii. аналогично для Кт.

38 <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=9409962354>

39 https://confluence.sberbank.ru/display/TPDCTOTCF/TSO_APP_SETTING_TRANSACTION_FILTER

40 https://confluence.sberbank.ru/display/TPDCTOTCF/TSO_APP_SETTING_TRANSACTION_FILTER

41 <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=7037422388>

42 <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=7037422388>

43 <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=7037422388>

44 <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=7037422388>

- iii. Для масок Дт и Кт сделать строку вида:
 - i. "СчетДт like <МаскаДт1> **or** СчетДт like <МаскаДт2> **or** ..."
 - ii. "СчетКт like <МаскаКт1> **or** СчетДт like <МаскаКт2> **or** ..."
- iv. Если заполнены обе маски Дт и Кт, сделать <условиеДт> **and** <условиеКт>, иначе только заполненное условие.
- b. Составить общее *условие фильтра* из строк п.1: (Условие_строка1) **or** (Условие_строка2) **or** ...

 Чтение настроечной таблицы и заполнение на ее основе "Условия фильтра" желательно вынести в отдельную функцию, чтобы можно было переиспользовать в дальнейшем для других настроечных таблиц.

d. Нам нужны поля:

- i. AccountingDocument (ad):
 - 1. id
 - 2. operationDate
 - 3. docdate
 - 4. docnumber
 - 5. paymentpurpose
 - 6. paymentpurposepublic
 - 7. ctl_operationdate_part
- ii. Transaction (t1 и t2)
 - 1. id
 - 2. operdate
 - 3. sum
 - 4. sumInval
 - 5. servid
 - 6. operationid
 - 7. productregister
 - 8. pairid
 - 9. productfabriccode
 - 10. account
 - 11. corrAccount
 - 12. debetFlag
 - 13. ctl_validfrom
 - 14. ctl_operdate_part
- iii. Account (a1 и a2)
 - 1. id
 - 2. number.

- 2. Определяем servid для дальнейшего джойна с УДКЗ:
 - a. Если t2.servid <> NULL, берем его.
 - b. Иначе берем t1.servid.
- 3. Определяем productRegister для дальнейшего джойна с УДКЗ:
 - a. Если t1.productRegister <> NULL, берем его.
 - b. Иначе берем t2.productRegister.
- 4. Мы получили датасет "Проводки".

- ❗ По итогам всех дальнейших преобразований количество записей в датасете не должно измениться:
- a. если что-то не приджойнилось, запись не исключаем.
 - b. возникающие дубли убираем.
 - i. Уникальный ключ проводки: `gbk.ad.id`⁴⁵ + `t1.pairid` (`t1.pairid=t2.pairid`).
 - ii. В тех местах, где ожидаются дубли - описаны условия выбора 1 записи.
 - iii. Если будут еще дубли - будем дополнять аналитику.

7.2 Добавим ID проводки в ЕКС/gl_gl для сверки с проводками для РНУ-4,5

Джойним с таблицей **EksGbkLink** - хотим получить ID проводки в ЕКС для сверки с проводками в `tax_transaction_rnu_4_5`⁴⁶ (Т1 для РНУ-4,5, сейчас заполняется по `gl_ndl_eks`).

- ❗ Сейчас у нас:
- в датасете: `gbk_document_id_ek = gbk.accountingDocument.id = trn_sid` из `p_gl_sl`
 - в `tax_transaction_rnu_4_5`⁴⁷: `transaction_id = trn_sid` из `gl_ndl_eks`.
 - Это разные SIDы.
 - Поэтому ищем `trn_sid` из `gl_ndl_eks`.
 - Когда переведем РНУ-4,5 на `gl_gl`, здесь будем получать `trn_sid` для `gl_gl`.

1. Джойним Датасет (left join) с EksGbkLink (далее **link**):
 - a. Условие джойна:
 - i. `link.gbkDebetTransactionId = t1.id` и -- джойним не по номеру документа, а по ID полупроводок Дт и Кт, чтобы избежать дублей в случае сложных документов.
 - ii. `link.gbkCreditTransactionId = t2.id` и
 - iii. `link.valid is true`
 - b. Фильтр для оптимизации:
 - i. `$start_dt <= link.ctl_operdate_part <= $end_dt` -- поле партицирования string, например '2025-02-13'
 - ii. фильтр добавлять до джойна (мы не должны потерять записи датасета, если не нашлось записи link)
 - c. Нам нужны поля link:
 - i. для сохранения в витрину:
 1. `eksID`
 - ii. только для выборки и джойна:
 1. `gbkDebetTransactionId`
 2. `gbkCreditTransactionId`
 3. `valid`
 4. `operdate`
2. Желательно сделать этот пункт "отключаемым" - пропускать его (по какой-то константе ?)  к разработчику.
 Можем решить не заполнять эту связь (хотелось бы без выпуска релиза):
 - a. если будет сильно ухудшаться производительность, либо

⁴⁵ <http://gbk.ad.id>

⁴⁶ https://confluence.sberbank.ru/display/PND/tax_transaction_rnu_4_5

⁴⁷ https://confluence.sberbank.ru/display/PND/tax_transaction_rnu_4_5

- б. если какие-то проблемы с репликой

7.3 Добавим сумму исправительных проводок

⚠ Пока этот пункт решили не делать.

- У описанного в этом пункте алгоритма поиска корректировок есть существенный недостаток: по окончании периода СПОД придется переформировывать заново всю витрину за прошлый год, иначе потеряем корректировки.
- Альтернатива есть: нужно включить в фильтр всю обратную корреспонденцию.
- Если добавить к проводкам связь из `trn_trn`, то по заполненной таким образом за год витрине можно будет построить "2й этаж" - там связать проводки из разных месяцев друг с другом по ссылкам из `trn_trn`.
- Пока решили отложить реализацию из-за наличия более приоритетных задач.
- Есть BR: [14.2025. Обратные проводки](#)⁴⁸. Позже в рамках него вернемся к этой теме.
- Пока **не делаем** то, что выделено зеленым.

- i** К проводке из нашей выборки могут быть заведены корректирующей проводки (от 0 до N).
- `trn_trn.correct_trn_sid` - id корректирующей проводки
 - `trn_trn.trn_sid` - id исправляемой (оригинальной)

Корректирующие проводки в нашу основную выборку не попадают (отфильтруются по маскам).

Даты корректирующих проводок - не раньше даты исходной проводки.

Здесь мы делаем следующее:

1. Готовим вспомогательный датасет "Корректировки" с полями:
 - a. id исходной проводки.
 - b. Сумма всех корректировок для этой проводки.
2. По left join присоединяем датасет "Корректировки" к основному датасету "Проводки".

Подробнее о выборе источника для поиска корректировок:

Исходный датасет "Проводки" мы формируем из источника `repl_gbk_stream`. В ЕСС это соответствует дата-продукту `p_gl_sl`.

Строго говоря, корректировки нужно было бы искать:

1. Либо в `repl_gbk_stream.correctionDetailDTO + AccountingDocumentDTO + ...`
2. Либо в `p_gl_sl.trn_trn + p_gl_sl.trn + ...`

Но оба способа очень ресурсоемкие, поскольку:

- В табличках `correctionDetailDTO` и `trn_trn` нет даты корректирующей проводки, поэтому партии по дате при дальнейшем отборе и джойне использовать не получится.
- Для того, чтобы исключить проводки переноса с одного символа на другой (70601/70601), нужно джойнить еще с полупроводками и счетами по 2 раза.

Поэтому применяем тот же алгоритм поиска корректировок, что используется для остальных алгоритмов ТЦО:

⁴⁸ <https://confluence.sberbank.ru/pages/viewpage.action?pagelId=16364405188>

1. Корректирующие проводки по доходам/расходам берем "готовые" из [tax_transaction_rnu_4_5](#)⁴⁹ (Т1 для РНУ-4,5, сейчас заполняется по gl_ndl_eks, в дальнейшем будет переход на gl_gl).
2. Связи trn_trn тоже берем из gl_ndl_eks. После перевода РНУ-4,5 на источник gl_gl здесь тоже надо будет перейти.

Для этого принимаем следующие временные ограничения:

1. Проводки по начислению комиссий ЮЛ не "сворачиваются" при записи в ЕКС (в дальнейшем gl_gl), пишутся как есть.
2. Корректировки могут быть только в сторону уменьшения суммы
 - a. Вообще говоря, это не так. Но разбираться с положительными корректировками будем чуть позже.
 - b. Если будет положительная корректировка, она попадет к нам дважды: как проводка в основную выборку, и + как корректировка.
3. Корректировки для сложных проводок пока не рассматриваем.
 - a. У нас в датасете "Проводки" только проводка по доходам/расходам
 - b. В [tax_transaction_rnu_4_5](#)⁵⁰ тоже только доходы/расходы
 - c. Пока будем считать, что, даже в случае сложной проводки, - проводка на счет 706 в ней будет одна. Поэтому проблем с дублями или неправильной суммой корректировки возникнуть не должно.
 - i. В gl_ndl_eks мы с такими проблемами пока не сталкивались.

1. Готовим вспомогательный датасет "Корректировки":

- a. Выбираем проводки из [tax_transaction_rnu_4_5](#)⁵¹ (далее **rnu_4_5_temp**):
 - i. Фильтр:
 1. `rnu_4_5_temp.input_dt >= $start_date` и
 2. `rnu_4_5_temp.input_dt <= 31.01.ГГГГ`, где ГГГГ = год(\$end_date) + 1 – корректировка может быть заведена позже окончания периода загрузки, но считаем, что не позже окончания периода СПОД в следующем году.
 3. `и not (rnu_4_5_temp.dt_account like '7%' and rnu_4_5_temp.kt_account like '7%')` (исключаем проводки перевода символа и т.п.)
 - b. Присоединяем (inner join) gl_eks.trn_trn:
 - i. Условие джойна
 1. `rnu_4_5_temp.id_pk = trn_trn.correct_trn_sid`
 - ii. Фильтр:
 1. `trn_trn.instance_code = 'eks'` -- поле партицирования
 2. `trn_trn.deleted_flag = false`
 - c. Группируем по id исходной проводки и вычисляем сумму корректировок
 - i. Поле группировки:
 1. `trn_trn.trn_sid`
 - ii. Итоги:
 1. `sum( rnu_4_5_temp.amount )` запишем как `corr_amt`
 2. `sum( rnu_4_5_temp.amount_in_ccy )` запишем как `corr_amt_in_ccy`.

2. Обогащаем основной датасет "Проводки" рассчитанной суммой корректировок:

⁴⁹ https://confluence.sberbank.ru/display/PND/tax_transaction_rnu_4_5

⁵⁰ https://confluence.sberbank.ru/display/PND/tax_transaction_rnu_4_5

⁵¹ https://confluence.sberbank.ru/display/PND/tax_transaction_rnu_4_5

- a. Присоединим (left join) к основному датасету "Проводки" вспомогательный датасет "Корректировки":
 - i. Условие джойна:
 1. "Проводки". link. EksID = "Корректировки". trn_trn. trn_sid

7.4 Ищем операции в реплике УДКЗ

i У нас в датасете есть проводки по счетам (в корреспонденции с 70601):

- **47423, 47422**
- 407, 408 (клиентские)
- 30301, 30302 (межфилиальные обороты)
- 301 - кор. счета

В УДКЗ, предположительно, должны найтись все операции по счетам 47423, 47422.

- Но фильтровать дополнительно по номеру счета пока не будем, попробуем приджойнить все - посмотрим, что получится.

Операции по остальным счетам будем искать либо в Биллинге, либо еще где-то.

1. Джойним датасет Проводки с таблицами реплики УДКЗ - по ID сервиса:

- a. Условия джойна:
 - i. left join udkz.product_service_ids **psi** on psi.service_id = Датасет.serviceId
 - ii. left join udkz.product **pr** on pr.product_id = psi.product_id
 - iii. left join udkz.register **reg** on reg.product_id = pr.product_id
 - iv. left join udkz.filter_element_attribute **fea** on fea.register_id = reg.register_id
- b. Фильтр:
 - i. reg.register_status = 'CLOSED' -- **⚠** возможно, потом уточню условие - пока пусть будет так. На известных кейсах это дает возможность не получить дубли. Позже исследую, не потеряем ли что-нибудь так. **Фильтр отключен**
 - ii. этот фильтр применяем в условии left джойна (не теряем проводку, если нет записи со статусом CLOSED)
 - iii. pr.original_product_id not like '40817%' and original_product_id not like '40820%', если счет не попадает, исключаем всю строку (40817 - счета резидентов ФЛ, 40820 - счета нерезидентов ФЛ, последних может потребоваться добавить в будущем, пока исключаем)
- c. Нам нужны поля
 - i. product_service_ids:
 1. service_id
 2. product_id
 - ii. product
 1. product_id
 2. original_product_id
 3. amount
 4. client_id
 - iii. register
 1. register_id
 2. product_id
 3. account_number
 4. begin_date

5. `ctl_validfrom`
 - iv. `filter_element_attribute`
 1. `register_id`
 2. `product_name`
 3. `product_factory_name`
 4. `comm_instance_id`
 5. `commission_code`
 - d. В результате этого джойна мы нашли операции для части проводок по счетам 47423, возможно еще по 47422.
 - i. Для остальных счетов ничего не приджойнится, поля УДКЗ оставим пустыми.
 - ii. Для некоторых счетов 47423 также можем не найти запись (это кривизна, но пока бывает).
 - iii. Может найтись более одной записи для Проводки.
 - iv. По информации от коллег из УДКЗ, счета 47422 в базу на данный момент не пишутся. Для связи с ними нужна реплика АЕ (`INTERNAL_AEACC_AEACC`). Из нее нужно по номеру счета отобрать активные связи и регистры, по которым взять данные из УДКЗ. При этом поле с регистром может джойниться через `register.old_register_id`, если в нем 19 символов, либо через `register.register_id`, если в нем 36 символов.
 1. Пока оставляем как есть. Будут дубли - будем разбираться.
2. Теперь джойним датасет с таблицами реплики УДКЗ по ID регистра
- ❗ На данных декабря 2024 работала связь по `service_id`. Но на данных апреля 2025 эта связь уже не работает. Теперь ищем по ID регистра. Будут дубли - надо их убрать.**

Дополнительно получена информация, что счета 47422 не пишутся в УДКЗ, для их присоединения по ID регистра нужно сначала присоединить таблицу `account` реплики АЕ (`INTERNAL_AEACC_AEACC`).
- a. Подготовительный шаг. Джойним датасет с таблицей реплики АЕ для счетов 47422, которых нет в УДКЗ, чтобы получить **ID регистра** по ним:
 - i. Присоединяем (`left join`) таблицу `account` реплики АЕ
 1. Условия джойна:
 - a. `account.account = dt_account_num`, если `kt_account_num` like '706%' или `account.account = kt_account_num`, если `dt_account_num` like '706%'
 - b. `account.register_open_module = 'УДКЗ'`
 - c. `account.update_date = Датасет.gbк_operation_date`
 2. Если прошел джойн с репликой АЕ, проставляем `pпрb_code = 'Бизнес-карта'`, `Датасет.transaction.productRegister = account.register_object_id`
 - b. Возвращаемся к соединению с репликой УДКЗ. Условия джойна:
 - i. `left join udkz.register reg2 on reg2.register_id = Датасет.transaction.productRegister`, если длина поля `Датасет.transaction.productRegister` составляет 36 символов
 - ii. `left join udkz.register reg2 on reg2.old_register_id = Датасет.transaction.productRegister`, если длина поля `Датасет.transaction.productRegister` составляет 19 символов
 - iii. `left join udkz.product pr2 on pr2.product_id = reg2.product_id`
 - iv. `left join udkz.filter_element_attribute fea2 on fea2.register_id = reg2.register_id`
 - c. Фильтр:
 - i. `reg2.register_status = 'CLOSED'` Фильтр отключен
 - ii. этот фильтр применяем в условии `left джойна` (не теряем проводку, если нет записи со статусом `CLOSED`)
 - iii. `pr2.original_product_id not like '40817%' and original_product_id not like '40820%'`, если счет не попадает, исключаем всю строку (40817 - счета резидентов ФЛ, 40820 - счета

нерезидентов ФЛ, последних может потребоваться добавить в будущем, пока исключаем)

- d. Нам нужны поля - те же, что описаны выше в п.1 для таблиц:
 - i. product
 - ii. register
 - iii. filter_element_attribute
 - e. Убираем дубли (тут дубли будут):
 - i. Если для проводки (уник. ключ проводки описан выше) нашлось несколько записей, выбираем 1 любую.
3. Для проводок из датасета определяем поля УДКЗ по найденным в пунктах 1 или 2 соответствиям:
- a. если нашлась запись в п.1, заполняем поля из нее.
 - b. иначе заполняем из найденного соответствия в п.2.
 - c. если не нашлось ничего, проводку не теряем, оставляем нуллы.

7.5 Получаем данные договора из УДКЗ

По некоторым операциям (пока не факт, что по всем) УДКЗ сохраняет у себя не только ID договора, но также номер и дату договора.

Запишем их в витрину и сравним с тем, что получаем по ID из продуктовой фабрики.

(Если полностью совпадут, потом можно будет убрать получение из фабрики.)

1. Джойним датасет с таблицей document из реплики УДКЗ:
 - a. Условие джойна:
 - i. left join udz.document doc on doc.product_id = pr.product_id
 - b. Фильтр:
 - i. пока не нужен
 - c. Нам нужны поля document:
 - i. product_id
 - ii. number - Номер договора
 - iii. begin_date - Дата договора
2. Убираем дубли. Если нашли несколько записей для одного product_id:
 - a. выбираем любую запись, у которой number не NULL (пока я таких дублей не видела, возможно потом уточним выбор или уберем вообще)
 - b. если у всех number = NULL, берем любую с датой begin_date не NULL.
 - c. если у всех NULL, или ничего не нашли - запись по product_id не исключаем. Кол-во записей в датасете должно остаться тем же.

7.6 Ищем операции в реплике Биллинга

- i** У нас в датасете есть проводки по счетам (в корреспонденции с 70601):
- 47423, 47422 - эти должны были найтись в УДКЗ
 - **407, 408** (клиентские)
 - **30301, 30302** (межфилиальные обороты)
 - **301** - кор. счета

В биллинге рассчитываем найти остальное, что не нашлось в УДКЗ.
Что не найдем - будем разбираться и дополнять алгоритм.

1. Джойним (left join) датасет Проводки с таблицей **billing.wo_operation** (далее **wo**) - операции списания комиссии
 - a. Условие джойна:
 - i. `t1.serviceld = wo.billingRequestId` и -- так будет несколько записей из wo
 - ii. `wo.remittanceInf is not NULL` и `<> 'none'` -- так должна остаться одна запись (но это не точно). Если будут дубли - уточним.
 - b. Нам нужны поля wo_operation:
 - i. для сохранения в витрину:
 1. eksID
 2. remittanceinf
 - ii. только для выборки и джойна:
 1. calcID
 2. billingRequestId
 3. documentOperationID
 4. registerid
 5. productCode
 6. typeCode
 7. accountingDate
 8. orderDate
 9. createDate
 10. amount
 11. ctl_validfrom
2. Для каждой операции списания ищем соответствующую операцию расчета комиссии (ее может не быть):
 - a. Джойним (left join) с таблицей **billing.calc_operation** (далее **calc**)
 - b. Условие джойна:
 - i. `wo.calcid = calc.calcid`
 - c. Теоретически, можем получить дубли: сначала посмотрим на них - потом подумаем, как убирать.
 - d. Нам нужны поля calc_operation:
 - i. clientEpklId - ЕПК ID клиента
 - ii. agreementId - ID договора, в рамках которого списывается комиссия
 - iii. calcdatetime - дата/время расчета.

7.7 Получаем данные договора из реплики ППРБ-источника или из ЕСС

В зависимости от того, из какой продуктовой фабрики пришла комиссия, получаем номер и дату договора из соответствующей реплики или дата-продукта ЕСС.

Этот пункт будет дополняться по мере проведения аналитики по новым источникам.

1. Если **udkz.fea.commission_code like 'ZD_COM%'** (ППРБ Зарплатные договоры), получаем номер и дату договора из реплики **zdfd**:
 - a. Запишем код ППРБ-источника:
 - i. `pprb_code = 'ЗД'` -- возможно потом уточню
 - b.  Следующий джойн можно пропустить на первом шаге: возможно, мы получим все номера и даты договора из УДКЗ, и этого будет достаточно. Дальше можем реализовать этот джойн и сравнить полученные результаты с данными УДКЗ.

с. Джойним (left join) с таблицей SalaryAgreementCard (далее **sac**):

i. Условие джойна:

1. `trim( substr( sac.contract, 63, 50 )) = udkz.reg.original_product_id` -- в поле contract длинный текстовый префикс. С 63ей позиции начинается ID. Длина ID точно не известна, 50 это с запасом - хвост из пробелов обрежем.
2. `sac.cardVersion <> '-1.0'`
3. `sac.versionBeginDate < $end_dt` -- тут <=, если обрежем время
4. `sac.versionEndDate > $end_dt` или `is NULL` -- и тут <=, если обрежем время
5. `sac.externalStatus in ('SA_RESOLVED', 'SA_ARCHIVED')`

❗ В таблице SalaryAgreementCard хранятся N версий для одного договора (любое изменение параметров договора или клиента сохраняется как отдельная строка в таблице).
 Подробнее про жизненный цикл зарплатного/социального договора:
<https://confluence.delta.sbrf.ru/pages/viewpage.action?pagelId=12321692867>

У версий есть дата/время начала и окончания действия версии - `versionBeginDate` и `versionEndDate`, но эти временные интервалы слегка пересекаются, поэтому можем получить дубли.

Дата/время хранится в формате string и выглядит, например, так:

- 2010-02-25T00:00+03:00[Europe/Moscow]
- 2011-06-10T00:00+04:00[Europe/Moscow]
- 2021-02-20T12:56:59.941+03:00[Europe/Moscow]

Пока предлагаю просто брать `substr(поле, 1, 10)` от всех дат.

Мы точно получим дубли, поскольку за 1 дату бывает несколько версий - но мы получили бы их в любом случае, даже не обрезая время (просто в меньшем количестве).

Дубли нужно будет убрать.

ii. Убираем дубли:

Если получили более одной записи по одному `udkz.pr.original_product_id`, то:

1. выбираем запись с максимальным значением `sac.cardVersion`
 - а. сравниваем "как числа", а не просто как строки, т.е. `'2.10' > '2.9'`.

iii. Нам нужны поля SalaryAgreementCard:

1. `contractNumber`
2. `contractBeginDate` (обрежем время, оставим только дату)
3. `contractClassification` (2 - зарплатный договор, 1 - социальный договор).

d. Запишем номер и дату договора из ППРБ-источника:

- i. `pprb_agreement_num = contractNumber`
- ii. `pprb_agreement_date = Дата( substr( contractBeginDate, 1, 10 ))`
- iii. `pprb_agreement_type_code =`
 1. `'salary'`, если `contractClassification = '2'`
 2. `'social'`, если `contractClassification = '1'`
 3. `contractClassification`, иначе.

2. Если ... (еще какие-то виды комиссий, которые начисляются по отдельным договорам, а не по договору РКО)

а. Запишем код ППРБ-источника:

- i. `pprb_code =` -- какие-то еще фабрики, где ищем соответствующие договоры

3. Если `dt_product_fabric_code = 'КЮЛ'` и `kt_product_fabric_code = 'КЮЛ'`:

- a. Запишем код ППРБ-источника:
 - i. `pprb_code = 'КЮЛ'`
 - ii. присоединяем кредитные договоры через номер счета (47441):
 - b. Джойним (left join) датасет с таблицей `gl_idl.acct` для `$end_dt < 01.10.2025`, с таблицей `gl_idl1.acct` для `$end_dt >= 01.10.2025`
 - i. Условия джойна:
 1. `acct.num = dt_account_num`, если `kt_account_num like '706%'` или `acct.num = kt_account_num`, если `dt_account_num like '706%'`
 2. `acct.instance_code = 'p_gl_sl'` для `gl_idl`, `acct.instance_code = 'p_gl_slacct-mono'` для `gl_idl1`,
 3. `acct.close_dt >= gbk.operation_date` или `acct.close_dt is null`
 - c. Джойним (left join) таблицу `gl_idl.registry_acct` для `$end_dt < 01.10.2025`, с таблицей `gl_idl1.registry_acct` для `$end_dt >= 01.10.2025`
 - i. Условия джойна:
 1. `registry_acct.acct_sk = acct.acct_sk`
 2. `registry_acct.deleted_flag = false`
 3. `registry_acct.instance_code = 'p_gl_prs'`
 - d. Джойним (left join) таблицу `corp_cgc_ndl_p_kul.registry`
 - i. Условия джойна:
 1. `registry.registry_sk = registry_acct.registry_sk`
 2. `registry.deleted_flag = false`
 - e. Джойним (left join) таблицу `corp_cgc_ndl_p_kul.agr` (если строк получится несколько, оставить одну любую)
 - i. Условия джойна:
 1. `agr.agr_sk = registry.agr_sk`
 2. `agr.close_dt >= gbk.operation_date` или `agr.close_dt is null`
 3. `agr.deleted_flag = false`
 4. `agr.type_code = 'corp_cred>line'`
 - ii. Нам нужны поля `agr`:
 1. `sid`
 2. `agr_dp_code`
 3. `num`
 4. `open_dt`
 5. `type_code`
 6. `crpcr_debtor_subj_sk`
 - f. Запишем данные договора из ECC:
 - i. `ess_agreement_dp_code = agr.agr_dp_code`
 - ii. `ess_agreement_ek = agr.sid`
 - iii. `ess_agreement_num = agr.num`
 - iv. `ess_agreement_date = agr.open_dt`
 - v. `ess_agreement_type_code = agr.type_code`
4. Иначе (для всего остального пока берем договор РКО из ECC (`settl_serv_org_idl_debug`, дата-продукт `settl_serv_org_ndl_p_rko_c_rko`):
- a. Джойним (left join) датасет с таблицей `agr`:
 - i. Условия джойна:
 1. `agr.sid = calc.agreementid`
 2. `agr.agr_dp_code = 'settl_serv_org_ndl_p_rko_c_rko'`
 - ii. Нам нужны поля `agr`:
 1. `sid`
 2. `agr_dp_code`
 3. `num`
 4. `open_dt`
 5. `type_code`.

- b. Запишем данные договора из ЕСС:
 - i. `ess_agreement_dp_code = agr.agr_dp_code`
 - ii. `ess_agreement_ek = agr.sid`
 - iii. `ess_agreement_num = agr.num`
 - iv. `ess_agreement_date = agr.open_dt`
 - v. `ess_agreement_type_code = agr.type_code`
- 5. Если `calc.agreementid` не заполнен, присоединяем договоры РКО напрямую через номер счета:
 - a. Джойним (left join) датасет с таблицей `gl_idl.acct` для `$end_dt < 01.10.2025`, с таблицей `gl_idl1.acct` для `$end_dt >= 01.10.2025`
 - i. Условия джойна:
 - 1. `acct.num = dt_account_num`, если `kt_account_num like '706%'` и `dt_product_fabric_code = 'РКОЮЛ'` или `acct.num = kt_account_num`, если `dt_account_num like '706%'` и `kt_product_fabric_code = 'РКОЮЛ'`
 - 2. `acct.instance_code = 'p_gl_sl'` для `gl_idl`, `acct.instance_code = 'p_gl_slacct-mono'` для `gl_idl1`,
 - 3. `acct.close_dt >= gbk.operation_date` или `acct.close_dt is null`
 - b. Джойним (left join) таблицу `gl_idl.registry_acct` для `$end_dt < 01.10.2025`, с таблицей `gl_idl1.registry_acct` для `$end_dt >= 01.10.2025`
 - i. Условия джойна:
 - 1. `registry_acct.acct_sk = acct.acct_sk`
 - 2. `registry_acct.instance_code = 'p_gl_prs'`
 - c. Джойним (left join) таблицу `settl_serv_org_idl_debug.registry`
 - i. Условие джойна:
 - 1. `registry.registry_sk = registry_acct.registry_sk`
 - d. Джойним (left join) таблицу `settl_serv_org_idl_debug.agr`
 - i. Условия джойна:
 - 1. `agr.agr_sk = registry.agr_sk`
 - 2. `agr.close_dt >= gbk.operation_date` или `agr.close_dt is null`
 - ii. Нам нужны поля `agr`:
 - 1. `sid`
 - 2. `agr_dp_code`
 - 3. `num`
 - 4. `open_dt`
 - 5. `type_code`
 - 6. `leg_ent_acct_owner_subj_sk`
 - e. Запишем данные договора из ЕСС:
 - i. `ess_agreement_dp_code = agr.agr_dp_code`
 - ii. `ess_agreement_ek = agr.sid`
 - iii. `ess_agreement_num = agr.num`
 - iv. `ess_agreement_date = agr.open_dt`
 - v. `ess_agreement_type_code = agr.type_code`

7.8 Добавляем ИНН, наименование и прочие данные клиента

- 1. Определяем ЕПК ID клиента:
 - a. `subject_ek =`
 - i. `udkz.product.client_id`, если не NULL.
 - ii. `billing.calc.clientEpkId`, если не NULL.
 - iii. `subj.sid` для данного `subj.subj_sk = agr.leg_ent_acct_owner_subj_sk` из `settl_serv_org_idl_debug`, если не NULL.
 - iv. `subj.sid` для данного `subj.subj_sk = agr.crpcr_debtor_subj_sk` из `corp_cgc_ndl_p_kul`, если не NULL иначе.

2. Джойним (left join) с таблицей **subj** (если потребуется добавление ФЛ, то для счетов 40817 и 40820 связь идет через **subj**, **subj_indidfn** и **subj_h** из дата-продукта **subj_ind_idl**):
 - a. Условие джойна:
 - i. если **subject_ek** из п.1 не NULL, присоединяем по **subj.sid = subject_ek** и **subj.deleted_flag is false**
 - ii. если **subject_ek** из п.1 NULL, присоединяем по **subj.subj_sk = acct.subj_sk** и **subj.deleted_flag is false**.
 - b. Нам нужны поля **subj**:
 - i. **subj_sk**
 - ii. **subj_dp_code**
 - iii. **cntry_sk**
 - iv. **cntry_dp_code**
 - v. **reg_cntr_reg_num**
 - vi. **swift_code**
 - vii. **international_cmpny_flag**.
3. Берем наименование клиента из таблицы **subj_orgidfn**:
 - a. Условие джойна (left join):
 - i. **subj.subj_sk = subj_orgidfn.subj_sk** и
 - ii. **subj.subj_dp_code = subj_orgidfn.subj_dp_code**
 - iii. **subj_orgidfn.start_dt <= \$end_dt <= subj_indidfn.end_dt** -- берем наименование на конец периода загрузки. ⚠ Все даты везде сравниваем как даты, без учета времени
 - b. Нам нужны поле **subj_orgidfn**:
 - i. **short_name**
 - ii. **international_name**
4. Берем ИНН клиента из таблицы **subj_h**:
 - a. Условие джойна (left join):
 - i. **subj.subj_sk = subj_h.subj_sk** и
 - ii. **subj.subj_dp_code = subj_h.subj_dp_code**
 - iii. **subj_h.start_dt <= \$end_dt <= subj_h.end_dt** -- берем запись на конец периода загрузки.
 - b. Нам нужно поле **subj_h**:
 - i. **inn_num**.
5. Получаем страну регистрации из таблицы **rdm_idl_spark.cntry**:
 - a. Условие джойна (left join):
 - i. **cntry.cntry_sk = subj.cntry_sk** и
 - ii. **cntry.cntry_dp_code = subj.cntry_dp_code**
 - b. Нам нужны поля **cntry**:
 - i. **digital_code**.

7.9 Добавляем данные о ВЗЛ и РОЗ

1. Джойним (left join) с датасетом **interdependent_entity_data** (ВЗЛ, см. п.6.7)
 - a. Условие джойна:
 - i. **replace(subj.reg_cntr_reg_num, ' ', '') = replace(interdependent_entity_data.inc_country_reg_number, ' ', '')** ⚠ (требуется удалить все пробелы из строк) и обе строки не NULL и не пустые **ИЛИ**
 - ii. **subj_h.inn_num = interdependent_entity_data.inn** и обе строки не NULL и не пустые **ИЛИ**
 - iii. **replace(subj.swift_code, ' ', '') = replace(interdependent_entity_data.swift_code, ' ', '')** ⚠ (требуется удалить все пробелы из строк) и обе строки не NULL и не пустые
 - b. ⚠ Если в полях, по которым джойним, стоит пустая строка (не NULL), соединение происходить не должно, иначе записи размножатся по равенству пустых строк ⚠

- c. Нам нужны поля `interdependent_entity_data`:
 - i. `code`
 - ii. `full_russian_name`
 - iii. `inc_country_reg_number`
 - iv. `inn`
 - v. `swift_code`
 - vi. `special_tax_regime_code.code`
 - vii. `special_tax_regime_code.description`
 - viii. `list_inclus_reason_date`
 - ix. `list_exclus_reason_date`
- 2. Джойним (left join) с датасетом **offshore_state_ter_data** (Оффшорные зоны, см. п.6.7)
 - a. Условие джойна:
 - i. `cntry.digital_code = offshore_state_ter_data.country_code`
 - b. Нам нужны поля `offshore_state_ter_data`:
 - i. `country_code`
 - ii. `effective_order_code.order_effective_date`
 - iii. `termination_order_code.order_effective_date`

7.10 Добавляем код филиала

Присоединим счет доходов/расходов из ECC (**gl_idl**) и возьмем из него ссылку на филиал:

- 1. Выберем ID счета доходов/расходов из проводки:
 - a. `gbk.a2.id`, если `gbk.a2.number` like '7%',
 - b. `gbk.a1.id`, иначе.
- 2. Джойним (left join) с таблицей **gl_idl.acct** для `$end_dt < 01.10.2025`, с таблицей **gl_idl1.acct** для `$end_dt >= 01.10.2025`:
 - a. Условие джойна:
 - i. `acct.sid` = ID счета доходов/расходов (из п.1)
 - ii. `acct.instance_code = 'p_gl_sl'` для **gl_idl**, `acct.instance_code = 'p_gl_slacct-mono'` для **gl_idl1** -- это поле партицирования
 - b. Нам нужны поля `acct`:
 - i. `acct_sk`
 - ii. `acct_dp_code`
- 3. Джойним (left join) с таблицей **gl_idl.acct_h** для `$end_dt < 01.10.2025`, с таблицей **gl_idl1.acct_h** для `$end_dt >= 01.10.2025`:
 - a. Условие джойна:
 - i. `acct_h.acct_sk = acct.acct_sk` и
 - ii. `acct_h.acct_dp_code = acct.acct_dp_code` и
 - iii. `acct_h.start_dt <= gbk.ad.operationDate <= acct_h.end_dt` ⚠ Все даты везде сравниваем как даты, без учета времени
 - b. Нам нужны поля `acct_h`:
 - i. `bal_div_sk`
 - ii. `bal_div_dp_code`
- 4. Джойним (left join) с датасетом **vspc** (заполнение датасета `vspc` см. п. 6.6):
 - a. Условие джойна:
 - i. `vspc.div_sk = acct_h.bal_div_sk`

- b. Нам нужны поля vspc:
 - i. terr_bank_subbranch_code
 - ii. terr_bank_div_sk.
- 5. Джойним (left join) с таблицей маппинга "Филиал источника - филиал витрин" TECHDM_FILIAL_MAP⁵² (далее **filial_map**):
 - a. Условие джойна:
 - i. filial_map.filial_ek = vspc.terr_bank_div_sk **и**
 - ii. filial_map.dp_code_fk = acct_h.bal_div_dp_code
 - iii. filial_map.start_date ≤ gbk.ad.operationDate ≤ filial_map.end_date.
 - b. Нам нужно поле filial_map:
 - i. filial_vdun_fk.

7.11 Искключаем ИП и НКЦ

Необходимо исключить строки, у которых (см. Константы⁵³):

- dt_account_num like \$commission_exclude_account_entrepreneur или
- kt_account_num like \$commission_exclude_account_entrepreneur или
- dt_account_num in \$commission_exclude_account_nko_nkc или
- kt_account_num in \$commission_exclude_account_nko_nkc.

7.12 Заносим в таблицу витрины

Заносим данные из датасета в таблицу витрины.

⁵² https://confluence.sberbank.ru/display/PND/TECHDM_FILIAL_MAP

⁵³ <https://confluence.sberbank.ru/pages/viewpage.action?pageId=6043895168>

8 Заполнение данных в таблице витрины

Раздел описывает шаг 3 процесса загрузки.

8.1 Заполнение полей таблицы

Поле	Описание	Алгоритм заполнения
gbk_operation_date	Дата проводки в ГБК	gbk.ad.operationDate
gbk_document_id_ek	ID документа (проводки) начисления комиссии в ГБК	gbk.ad.id
eks_document_id_ek	ID документа (проводки) начисления комиссии в ЕКС (=trn_sid в ndl_eks)	link.EksID
gbk_document_num	Номер документа в ГБК (номер мемоордера)	gbk.ad.docNumber
gbk_transaction_amt	Сумма проводки в ГБК в базовой валюте	gbk.t2.sum
dt_gbk_transaction_amt	Сумма полупроводки Дт в валюте счета Дт (ГБК)	gbk.t1.sumInVal
kt_gbk_transaction_amt	Сумма полупроводки Кт в валюте счета Кт (ГБК)	gbk.t2.sumInVal
correction_amt	Сумма корректировок в базовой валюте	- corr_amt (см. п. 7.3)
correction_ccy_amt	Сумма корректировок в валюте счета, корреспондирующего с доходом/расходом	- corr_amt_in_ccy (см. п. 7.3)
corrected_transaction_amt	Сумма проводки с учетом корректировки	gbk_transaction_amt + nvl(correction_amt, 0)
dt_account_num	Номер счета Дт	gbk.a1.number
kt_account_num	Номер счета Кт	gbk.a2.number

Поле	Описание	Алгоритм заполнения
dt_account_gbk_ek	ID счета Дт в ГБК	gbk.t1.account
kt_account_gbk_ek	ID счета Кт в ГБК	gbk.t2.account
dt_product_fabric_code	Код продуктовой фабрики полупроводки Дт	gbk.t1.productFabricCode
kt_product_fabric_code	Код продуктовой фабрики полупроводки Кт	gbk.t2.productFabricCode
gbk_complex_document_flag	Флаг сложной проводки в ГБК	gbk.ad.complexDocument
gbk_transaction_pair_id_ek	ID пары полупроводок в ГБК	gbk.t1.pairid
payment_purpose_public_txt	Назначение платежа из учетного документа (публичная часть)	gbk.ad.paymentPurposePublic
payment_purpose_txt	Назначение платежа из учетного документа (маскированная часть)	gbk.ad.paymentPurpose
dt_service_id_ek	ID сервиса полупроводки Дт	gbk.t1.serviceId
kt_service_id_ek	ID сервиса полупроводки Кт	gbk.t2.serviceId
dt_product_register_ek	ID продуктового регистра Дт	gbk.t1.productRegister
kt_product_register_ek	ID продуктового регистра Кт	gbk.t2.productRegister
udkz_product_id_ek	ID продукта для операции начисления комиссии в УДКЗ	udkz.product.product_id
udkz_account_num	Номер счета в УДКЗ	udkz.register.account_number
udkz_amount_amt	Сумма начисления в УДКЗ	udkz.product.amount
udkz_product_name_txt	Наименование продукта в УДКЗ	udkz.fea.product_name

Поле	Описание	Алгоритм заполнения
udkz_product_factory_name_txt	Наименование продуктовой фабрики в УДКЗ	udkz.fea.product_factory_name
udkz_commission_code	Код комиссии в УДКЗ	udkz.fea.commission_code
udkz_comm_instance_id_ek	Сквозной ID услуги в УДКЗ	udkz.fea.comm_instance_id
udkz_client_ek	ЕПК ID клиента в УДКЗ	udkz.product.client_id
udkz_agreement_ek	ID договора в УДКЗ	udkz.product.original_product_id
udkz_agreement_num	Номер договора в УДКЗ	udkz.document.number
udkz_agreement_date	Дата договора в УДКЗ	udkz.document.begin_date
udkz_registry_status	Статус регистра в УДКЗ	udkz.reg.register_status
billing_product_code	Код продукта/услуги в биллинге	wo.productcode
billing_request_id_ek	ID запроса на списание комиссии в биллинге	wo.billingrequestid
billing_client_ek	ЕПК ID клиента в биллинге	calc.clientEpkId
billing_agreement_ek	ID договора в биллинге	calc.agreementid
billing_accounting_date	Дата учета в биллинге	wo.accountingdate
billing_order_date	Дата документа в биллинге	wo.orderdate
billing_create_ts	Дата/время создания записи о списании в биллинге	wo.createdate
billing_calc_ts	Дата/время расчета в биллинге	calc.calcdatetime
billing_operation_amt	Сумма операции в биллинге	wo.amount
billing_currency_code	Цифровой код валюты операции в биллинге	wo.currencycode

Поле	Описание	Алгоритм заполнения
billing_remittance_inf_txt	Назначение платежа в биллинге	wo.remittanceinf
pprb_code	Код ППРБ-источника (код продуктовой фабрики, породившей комиссию)	см. п. 7.7
pprb_agreement_num	Номер договора в ППРБ-источнике	
pprb_agreement_date	Дата договора в ППРБ-источнике	
pprb_agreement_type_code	Тип договора в ППРБ-источнике	
ess_agreement_dp_code	Код дата-продукта ЕСС для договора	см. п. 7.7
ess_agreement_ek	ID (sid) договора в ЕСС	
ess_agreement_num	Номер договора в ЕСС	
ess_agreement_date	Дата договора в ЕСС	
ess_agreement_type_code	Тип договора в ЕСС	

Поле	Описание	Алгоритм заполнения
agreement_ek	ID договора в системе-источнике (sid)	Это данные договора, в рамках которого начислена комиссия. Этот договор дальше будем использовать в отчетах. Эти поля заполняем на основании заполненных выше данных договора, полученных из УДКЗ, из биллинга, из ППРБ или ЕСС. Тут логика выбора может меняться, если договор найдем в 2х и более местах. Заполнение agreement_type_code потом думаю привести к какой-то единой системе. 1. Если udkz_agreement_ek и udkz_agreement_num - оба не NULL, то: a. agreement_ek = udkz_agreement_ek b. agreement_num = udkz_agreement_num c. agreement_date = udkz_agreement_date d. agreement_type_code = udkz_commission_code -- ВОЗМОЖНО ПОТОМ ИЗМЕНИМ 2. Иначе если udkz_agreement_ek не NULL, то: a. agreement_ek = udkz_agreement_ek b. agreement_num = pprb_agreement_num c. agreement_date = pprb_agreement_date d. agreement_type_code = pprb_agreement_type_code 3. Иначе если ess_agreement_ek не NULL, то: a. agreement_ek = ess_agreement_ek b. agreement_num = ess_agreement_num c. agreement_date = ess_agreement_date d. agreement_type_code = ess_agreement_type_code

Поле	Описание	Алгоритм заполнения
agreement_num	Номер договора	4. Иначе (номер и дату для договора мы не нашли, но возможно хоть ID запишем): а. agreement_ek = billing_agreement_ek
agreement_date	Дата договора	
agreement_type_code	Тип договора	
subject_ek	ID клиента в системе-источнике (sid)	см. п. 7.8
subject_type_code	Субтип контрагента	subj.type_code (Отрезаем значение type_code = 'org>ent')
subject_name	Наименование клиента	coalesce(subj_orgidfn.short_name, subj_orgidfn.international_name)
subject_reg_num	Регистрационный номер клиента в стране инкорпорации	subj.reg_cntr_reg_num
subject_inn_code	ИНН клиента	subj.h.inn_num
subject_swift_code	SWIFT код клиента	subj.swift_code
subject_country_code	Страна регистрации клиента	cntry.digital_code
subject_international_cmpny_flag	Признак международной компании	subj.international_cmpny_flag

Поле	Описание	Алгоритм заполнения
subject_type	Тип клиента	- Если interdependent_entity_data.code is not null И offshore_state_ter_data.country_code is not null, то 'ВЗЛ' - Если interdependent_entity_data.code is not null И offshore_state_ter_data.country_code is null, то 'ВЗЛ' - Если interdependent_entity_data.code is null И offshore_state_ter_data.country_code is not null, то 'ПОЗ' - Если subj.international_cmpny_flag is TRUE, то 'МКО' (русские буквы) - иначе NULL
vzl_subject_name	Полное наименование клиента в справочнике ВЗЛ	interdependent_entity_data.full_russian_name
vzl_subject_reg_num	Регистрационный номер клиента в стране инкорпорации в справочнике ВЗЛ	interdependent_entity_data.inc_country_reg_number
vzl_subject_inn_code	ИНН клиента в справочнике ВЗЛ	interdependent_entity_data.inn
vzl_subject_swift_code	SWIFT код клиента в справочнике ВЗЛ	interdependent_entity_data.swift_code
vzl_subject_special_tax_regime_code	Режим налогообложения ВЗЛ. Код	interdependent_entity_data.special_tax_regime_code.code
vzl_subject_special_tax_regime_text	Режим налогообложения ВЗЛ. Описание	interdependent_entity_data.special_tax_regime_code.description
vzl_subject_include_date	ВЗЛ. Дата включения в список	interdependent_entity_data.list_inclusion_reason_date

Поле	Описание	Алгоритм заполнения
vzl_subject_exclude_date	ВЗЛ. Дата исключения из списка	interdependent_entity_data.list_exclus_reason_date
oz_subject_include_date	ОЗ. Дата включения в список	effective_order_code.order_effective_date
oz_subject_exclude_date	ОЗ. Дата исключения из списка	termination_order_code.order_effective_date
filial_ek	ID филиала в системе-источнике (sid)	vspc.terr_bank_div_sk  Для vspc_idl в качестве filial_ek используем sk (не sid) - т.к. sid'a как такового там нет, в поле sid пишется subbranch_code
filial_code	Филиал - бизнес-код в системе-источнике	vspc.terr_bank_subbranch_code
vdun_filial_code_long	Филиал - код в витринах данных (тип long)	filial_map.filial_vdun_fk
gbk_ctl_validfrom_ts	Дата-время актуальности в ГБК	gbk.t2.ctl_validfrom
udkz_ctl_validfrom_ts	Дата-время актуальности в реплике УДКЗ	udkz.register.ctl_validfrom
billing_ctl_validfrom_ts	Дата-время актуальности в реплике биллинга	billing.wo.ctl_validfrom
dp_code	Код дата-продукта - источника данных	'repl_gbk_stream'
partition_date	Дата комиссии для партицирования	gbk_operation_date

8.2 Запись данных в таблицу отчета

Сохранить данные датасета в таблицу [TAXDM_STG_COMMISSION](https://confluence.sberbank.ru/display/PND/TAXDM_STG_COMMISSION)⁵⁴ с полной заменой данных по \$Дата-продукту и \$Системе за период загрузки:

⁵⁴ https://confluence.sberbank.ru/display/PND/TAXDM_STG_COMMISSION

1. Удалить данные из таблицы витрины по условию:
 - a. `dp_code = $Дата-продукт`
 - b. `asnu_code = $Система`
 - c. `ГГГГММ( start_dt ) ≤ partition_date ≤ ГГГГММ ( end_dt )`
2. Записать строки датасета в таблицу.

9 Требования к нефункциональным характеристикам

9.1 Требования к журналированию

Загрузка данных должна протоколироваться в журнале событий, в том числе события:

- старт процесса загрузки
- завершение процесса загрузки
- чтение данных из таблиц источника

Подробнее см. [Журналирование и Аудит](#)⁵⁵.

9.2 Требования к частоте и периодичности загрузки

Загрузка таблицы должна проводиться:

- по расписанию: ежемесячно.
- по требованию: ручная загрузка (период указывается при запуске)

9.3 Требование к объему и времени загрузки

Прогнозируемый объем записей:

Время загрузки данных за квартал не должно превышать  часов для прогнозируемых объемов

9.4 Требование к первой загрузке данных

Предполагается заполнить данные начиная с 01.11.2024 или 01.12.2024 - для сверки с имеющимися эталонами 6.4

Промышленная эксплуатация - на данных 2025 года.

⁵⁵ <https://confluence.sberbank.ru/pages/viewpage.action?pageId=2329643287>